An AI-Assisted System for Designing, Deploying and Managing Networking Labs in Education

Ly Tuyet Trinh, Pham Duong Hoang Phu, Tran Anh Tuan, Phan Van Vinh *

School of Computing and Information Technology, Eastern International University, HCM City, Vietnam

trinh.ly.cit23@eiu.edu.vn, phu.phamduong.cit23@eiu.edu.vn, tuan.tran.cit23@eiu.edu.vn, vinh.phan@eiu.edu.vn

(*) Corresponding author

Abstract: Practical exercises are central to computer networking courses, yet students often struggle with device configuration, command syntax and troubleshooting, while instructors cannot follow every lab in real time. General-purpose AI assistants can explain concepts, but they do not see the network being configured, and letting them execute commands is unsafe. This paper presents NetAI, an AI-assisted system that connects a large language model (LLM) to real Cisco and Palo Alto devices in PNETLab and covers the whole lab life cycle: design, deployment, monitoring, configuration and recovery. An LLM-based intent router sends each chat message to a chat, read, configure or design pipeline; a design advisor turns a plain-language request into a validated Layer 2/3 design that can be exported as a new PNETLab lab and bootstrapped over the device console. The AI only proposes: the backend checks roles, scoped grants, command policy and automatically classified risk, and every change passes through Preview, Approve, Backup, Apply and Verify, with checkpoint-based undo. The model never receives credentials or management addresses. The implementation passes 1,930 automated tests, and the evaluation reports [router accuracy X%, design validity Y%] and no unsafe command reaching a device.

Keywords: Large Language Models, Network Management, Networking Education, Human-in-the-Loop, Intent-Based Networking, PNETLab.

I. INTRODUCTION

Computer networking is an important subject in information technology. Networking courses combine lectures with practical exercises in which students configure and troubleshoot IP addressing, VLANs, routing, switching and network security. This hands-on work is essential but difficult for beginners: a syntax error, an incorrect IP address or a wrong command can stop a network from working, and students must then examine the configuration to find the cause. In a class of many students, instructors cannot inspect every lab at the moment a problem appears.

Various intelligent systems have been introduced to support networking education. Budianto et al. (2019) developed an intelligent tutoring system that provided learning materials and exercises based on students' needs, and Verdú et al. (2017) used an intelligent tutoring system in a network design course to give feedback during learning. Ahmed and Hasnine (2023) found that general chatbots such as ChatGPT give detailed answers that do not always match the expected course content. Md Enzai (2024) studied ChatGPT as a learning tool in an introductory networking course, and Soto et al. (2024) integrated ChatGPT into an interactive environment for networking lessons. These studies reported improvements in student learning, but also issues with response quality, and they focus on learning content rather than on the devices students actually configure.

LLMs have also been applied to network configuration and automation. Wang et al. (2024) introduced NetConfEval to evaluate LLM-based configuration, Lira et al. (2025) proposed LLM-NetCFG, which translates natural-language intent into configurations and applies them with limited human intervention, and Provvedi et al. (2026) proposed Intent-LLM, which generates executable Python code for configuration and optimisation. Frameworks such as NAPALM support automated

operations across vendors (NAPALM Automation, 2026). Jiang et al. (2025) developed a zero-touch configuration framework for Time-Sensitive Networking, and Kim et al. (2026) surveyed LLM-based multi-agent systems for autonomous network management, identifying latency and reliability as open challenges. These systems aim at maximum automation; a teaching laboratory needs the opposite emphasis: human control, role separation between instructors and students, and the ability to undo mistakes.

Two gaps follow. First, an assistant that only reads a student's description does not know the current state of the network; a connectivity problem may come from an interface, a VLAN, the routing table or another setting. Connecting the assistant to the devices gives it current configuration and operational data. Second, applying AI-generated commands directly is risky: an incorrect command can interrupt connectivity. Configuration changes must therefore follow a controlled process with approval, command restrictions, backups, verification, audit logs and recovery. In addition, no existing system links an AI assistant to the full life cycle of an educational lab, from drawing the topology and building the lab to configuring, grading and restoring it.

This paper presents NetAI, which addresses these gaps. Its contributions are: (1) an architecture in which *the AI proposes and the backend disposes*, with an LLM-based intent router and server-side policy enforcement; (2) an AI design advisor and topology canvas connected to PNETLab, so that a plain-language description becomes a running, SSH-manageable lab; (3) a governance model for education, with Admin, Teacher and Student roles, scoped and expiring grants, automatic risk classification, approval digests and checkpoint-based undo; and (4) a quantitative evaluation together with scenarios on a real PNETLab laboratory. Section II describes the system design, Section III the implementation and evaluation, and Section IV concludes.

II. SYSTEM DESIGN

A. Design Principles

Five principles guide the design, and each is enforced by code rather than by prompt instructions. **P1, AI proposes, backend disposes:** the model classifies requests and suggests structured actions, but never opens an SSH session or decides permissions. **P2, least context:** the model receives hostnames, device kinds and a topology derived from parsed data, never credentials, management IP addresses or a full running configuration. **P3, deny by default:** unknown read-only commands are refused, and a permission grant is evaluated as one complete scope, never as a union of partial grants. **P4, every write is reversible and auditable:** a backup is taken before each change, verification runs after it, a checkpoint allows undo, and audit records are redacted. **P5, project isolation:** devices, chat sessions, changes, audit records and AI context of one project never reach another.

B. System Architecture

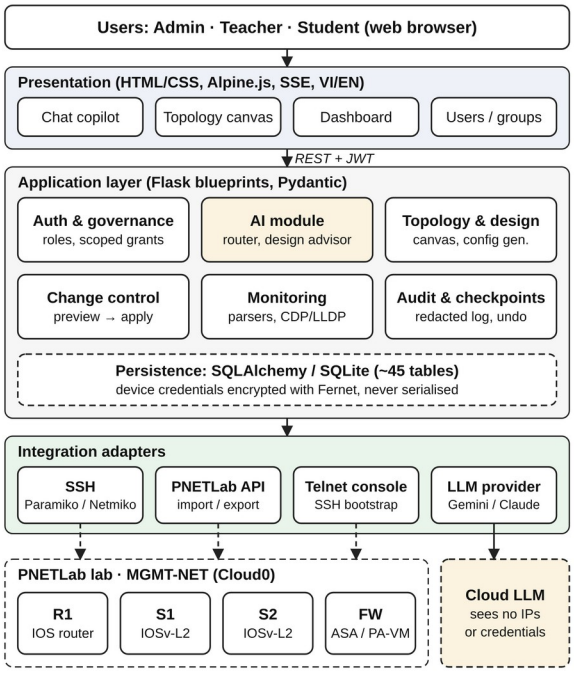


Figure 1. Proposed system architecture.

Figure 1 shows the four layers of the system. The presentation layer offers four pages, a chat copilot, a topology canvas, a dashboard and user/group management, built with HTML, CSS, JavaScript and

Alpine.js (Porzio, 2026), available in Vietnamese and English, and updated through server-sent events (SSE). The application layer is a Flask (Pallets, 2026) application with blueprints for authentication and governance, AI, topology and design, change control, monitoring, and audit and checkpoints; it exposes 176 REST endpoints, validates requests with Pydantic (Colvin, 2026), and stores data through SQLAlchemy (SQLAlchemy, 2026) in SQLite (SQLite Consortium, 2026), with schema changes managed by Flask-Migrate (Grinberg, 2026). Device credentials are encrypted with Fernet and never serialised. The integration layer contains the SSH adapters, Paramiko (Forcier, 2026) for Cisco IOS/IOSv-L2 and ASA and Netmiko (Byers, 2026) for Palo Alto PAN-OS, a PNETLab API client, a telnet console client and the LLM providers. Monitoring runs on APScheduler (Gronholm, 2026), and CLI output is parsed into interfaces, routes, OSPF neighbours, VLANs, trunks, ACLs and DHCP pools. Table 1 lists the main tools.

Table 1. Main tools and their roles in the system.

Tools	Technology	Role
Frontend	HTML/CSS, JavaScript, Alpine.js, SSE	Chat, topology canvas, dashboard; VI/EN interface
Backend	Python 3.11+, Flask 3	REST APIs, application factory, blueprints
Data	SQLAlchemy 2, Flask-Migrate, SQLite	~45 tables, 30 versioned migrations
Security	Flask-JWT-Extended, Fernet, Flask-Limiter	Token revocation, encrypted credentials, rate limits
Validation	Pydantic 2	API requests and AI output schemas
Network access	Paramiko, Netmiko, telnet console	SSH to Cisco IOS/ASA and PAN-OS; console bootstrap
Lab platform	PNETLab session API (requests)	Import, export and start labs
Monitoring	APScheduler	Bounded parallel device polling
AI	Google Gemini (default) / Anthropic Claude	Intent routing, structured actions, design proposals
Testing	pytest, fake SSH and AI providers	1,930 automated tests

C. AI Pipeline

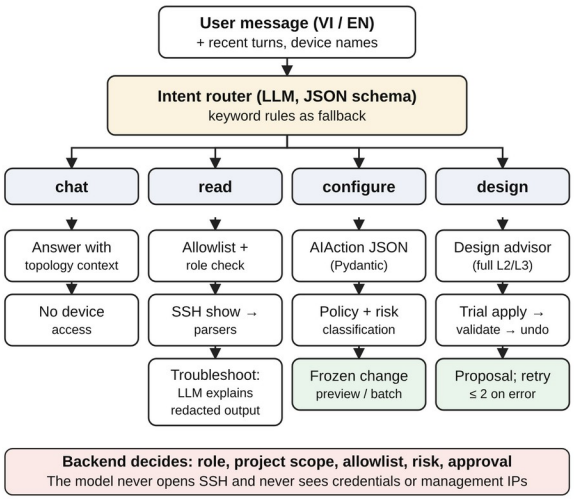


Figure 2. AI pipeline: intent routing and the four processing branches.

An AI agent is a program that performs tasks for a user by means of the tools available to it (Russell & Norvig, 2021). In NetAI the agent's tools are deliberately limited, and its output is always data that the backend checks. Figure 2 shows the pipeline.

Intent routing. Keyword rules cannot follow a real conversation in which a design, a read command, a configuration and a theory question follow one another. For example, after the assistant has suggested a design, "đặt dải IP là 172.20.5.0/24" (set the address range to 172.20.5.0/24) continues that design, whereas "chạy show running config trên SW1" reads a real device. A short LLM call therefore labels each message as *design*, *read*, *configure* or *chat*, using the message, the last six conversation turns, the project's device names and kinds, and whether the previous reply was a design. The answer must follow a JSON schema; if the call fails, the earlier keyword rules are used. The label only selects a pipeline: every pipeline still checks the user's role and project access.

Structured actions. For read and configure requests the model returns an AIAction object, validated with Pydantic, whose intent is monitor, configure, troubleshoot or chat and whose operations name the target hostname, execution mode, commands and verification commands. A chat answer carries no operation and never contacts a device. Read commands are checked against an allowlist before any SSH session is opened; for troubleshooting, the

output is redacted and sent back to the model to explain likely causes and next checks. A configure action is turned into a frozen, device-scoped change preview (Section II.E).

Safe context. The context sent to the model contains the project's devices (hostname, role, kind, status), the supported commands and a topology derived from monitoring snapshots, such as which VLAN carries which subnet and where its gateway is. Networks containing management addresses are removed. Instructors may add AI templates with course-specific guidance; templates enter the prompt but do not bypass any permission check.

Design advisor. For a design request, the model returns the complete Layer 2/3 design it wants the project to have: VLANs, access ports, SVIs, loopbacks, subinterfaces, static routes, single-area OSPF and optionally the addressing pool. The backend applies this proposal to the database inside a transaction, reads it back in normalised form (addresses marked "auto" become real ones), runs the design checks and then rolls the transaction back. If validation fails, the errors are returned to the model, which may try again (two attempts in total). Only a validated proposal is shown to the user, and accepting it changes the design, not the devices.

D. Network Design and Lab Provisioning

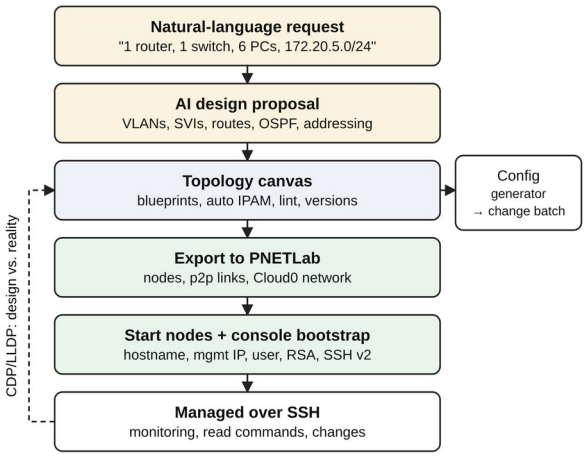


Figure 3. From a plain-language request to a managed PNETLab lab.

Each network is a project with its own management range, devices, topology, chat and change history. The topology canvas works like a network simulator: users drag routers, Layer 2/3 switches and firewalls, which are managed over SSH, as well as PCs, servers, clouds, access points and shared segments, which are drawn and addressed but never contacted. A managed device can be added as *planned* and becomes *provisioned* once it has a management address and login. Blueprints (star, branch, campus, DMZ, ring and hub-and-spoke) build a whole topology in one step, routed links and SVIs can take addresses automatically from the project's pool, and a design check reports problems such as overlapping subnets or unused VLANs. Versions of the diagram can be saved, compared and restored.

The configuration generator turns the design into commands per device, in the order VLANs, access ports, link interfaces, SVIs, static routes and OSPF for Cisco IOS, and routed interfaces (nameif, security level, address) and static routes for ASA. Port-channels, loopbacks and router-on-a-stick subinterfaces are supported. The generated commands are frozen into a change batch and follow the same workflow as any other change.

NetAI connects the design to PNETLab in both directions (Figure 3). An existing lab can be imported through PNETLab's session API. Conversely, a project can be exported as a new lab: each device becomes a node of the matching template at its canvas position, each cable a point-to-point link, and the management side of each managed device is connected to a Cloud0 network. A build that fails part way deletes what it created. The system then starts the nodes and, because a fresh node has no address or SSH, configures each one through its telnet console: hostname, management address, user, RSA keys, SSH version 2 and VTY access for Cisco IOS, and the management address, account and commit for Palo Alto PA-VM, which runs as a background job because a PA-VM needs ten to fifteen minutes to boot. After this step every device is managed over SSH. Finally, CDP/LLDP discovery compares the cabling reported by the devices with the design and shows matching, missing and undocumented links.

E. Safety-Governed Change Control

Roles and grants. NetAI distinguishes Admin, Teacher and Student accounts (Operator and Viewer access also exist inside shared projects). Teachers

create groups and student accounts; an Admin or Teacher grants actions such as config.create, config.approve, config.deploy, config.restore, device.observe and ai.use, scoped by group, device, environment and expiry. A grant may be marked delegable, and a delegated grant can never be wider than its parent; it lapses when the parent expires or is revoked, or when the student leaves the group. Students cannot receive approval rights.

Automatic risk classification. The backend, not the AI or the user, assigns each change a Low or High risk (Table 2). One High command makes the whole change High, and forbidden commands are rejected immediately rather than turned into a request for approval.

Table 2. Automatic risk classification of configuration changes.

Configuration group	Risk
Hostname, interface description, create/name VLAN, access port and access VLAN, in an isolated virtual lab on a non-critical device	Low
Routing, static routes, OSPF, ACLs, interface or management IP	High
Accounts, passwords, AAA, SSH/crypto, SNMP	High
Deleting configuration, shutdown, trunks, any command outside the Low set	High
Any change on production, core, distribution, ISP or firewall devices, or on a non-isolated lab	High
Restoring a configuration	High

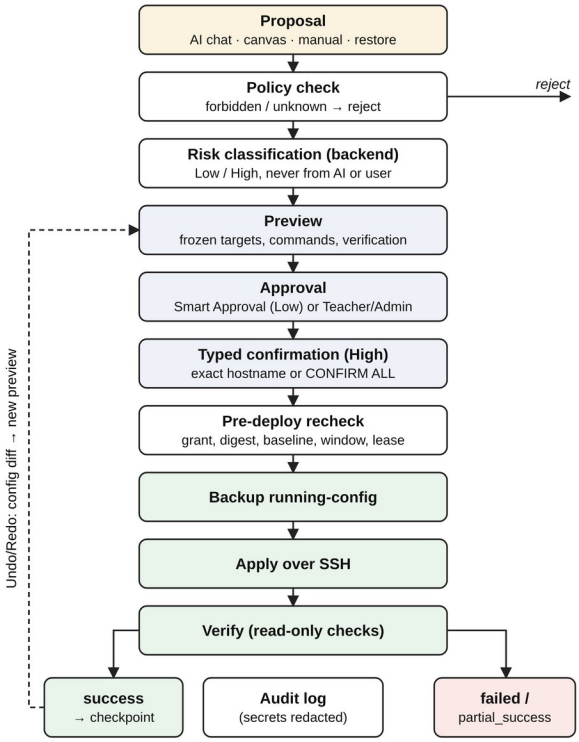


Figure 4. Safety-governed change workflow with checkpoint-based undo.

Workflow. Figure 4 shows the change workflow. A proposal from the chat, the canvas, a manual entry or a restore is checked against the command policy, classified, and frozen into a preview that fixes the target device identity, commands and verification. Low-risk changes in a group with Smart Approval enabled may be approved automatically; otherwise an authorised Teacher or the Admin must approve the exact configuration, and by default a Teacher cannot approve their own change. The approval is bound to a digest of the commands, verification, device, group, assignment, environment, baseline and policy, and expires after 60 minutes. High-risk changes also require typed confirmation of the exact hostname, or CONFIRM ALL for a batch. Immediately before SSH, the backend rechecks the grant, digest, baseline, maintenance window and device reservation, and takes a per-device lease that prevents two workers from writing to the same device. It then backs up the running configuration, applies the commands, runs read-only verification and records the result as success, failed or partial_success. A failed verification is never reported as success, and no rollback is executed automatically.

Checkpoint-based undo. Every change that applies successfully becomes a checkpoint holding the running configuration captured just before it. To undo or redo a step, the system reads the current configuration and computes the lines that restore the checkpoint's state by a structural comparison: lines only in the current configuration are negated, lines only in the target are added, and sections such as interface, router ospf or vlan are compared line by line regardless of order. Secrets, banners and certificates are never emitted and are reported as skipped. The result is stored as an ordinary preview and goes through the same approval, confirmation, backup and verification steps.

Protection of data. User passwords are hashed and device credentials are encrypted; changing a password, role or account status revokes all earlier tokens. Audit entries record who did what on which device and with what result, after secrets have been masked. Login attempts are rate limited and responses carry standard security headers.

F. Educational Features

Beyond device management, NetAI supports the organisation of a course. Teachers invite students to groups by code or invitation, publish assignments bound to a device, a time window and allowed command prefixes, and grade submissions from 0 to 100 with feedback. Students keep versioned drafts of their configurations, compare versions, validate them against the policy and submit one version. Device reservations prevent two groups from using the same physical device at the same time, a daily AI quota limits model usage, and in-platform notifications inform users of approvals and expiring permissions. These features distinguish NetAI from configuration-automation systems such as LLM-NetCFG or Intent-LLM, which do not address roles or teaching workflows.

III. IMPLEMENTATION AND EVALUATION

A. Test-bed

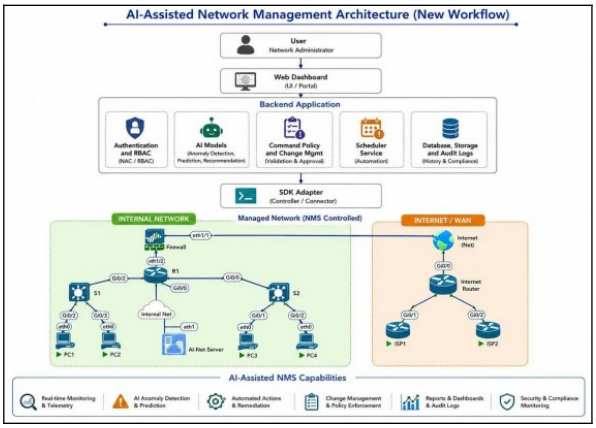


Figure 5. Evaluation test-bed.

Figure 5 shows the test-bed built on the course's PNETLab server, in the folder /Courses/CSN301-StudentLab/Group-1. Router R1 routes between two access switches, S1 and S2, which serve the PCs. The AI server runs the Flask backend under Gunicorn. Based on Table 3, R1, S1, S2 and the AI server are connected to the management network MGMT-NET (172.16.3.0/24, Cloud0), through which the backend monitors and manages the devices. [Add the firewall node and its address if a PA-VM or ASA is used in the final experiments.]

Table 3. Management IP address plan.

Device	Image / access	IP address	Device role
AI-SERVER	Linux, Gunicorn	172.16.3.28/24	NetAI backend
R1	Cisco IOSv, SSH (Paramiko)	172.16.3.111/24	Internal router
S1	Cisco IOSv-L2, SSH (Paramiko)	172.16.3.121/24	Access switch
S2	Cisco IOSv-L2, SSH (Paramiko)	172.16.3.122/24	Access switch

B. Functional Scenarios

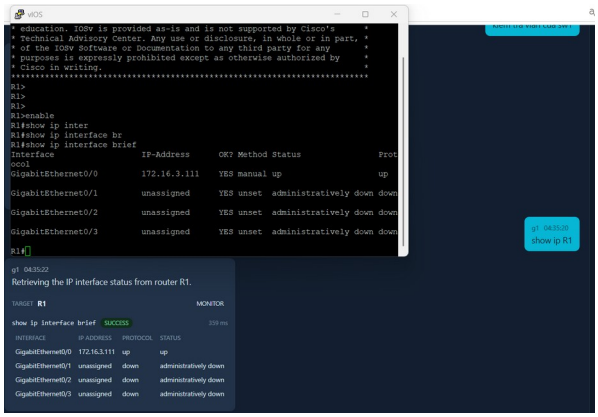


Figure 6. Retrieval of IP interface information from router R1.

Figure 6 shows a read request. When the user enters "show IP R1", which is not a valid Cisco command, the intent router labels it as a read request, the model identifies R1 and chooses show ip interface brief, the backend checks the command against the allowlist, connects to R1 over SSH and parses the output into a table of interfaces, addresses, status and protocol. Figure 7 shows the same process for "show vlan sw1" on a switch.

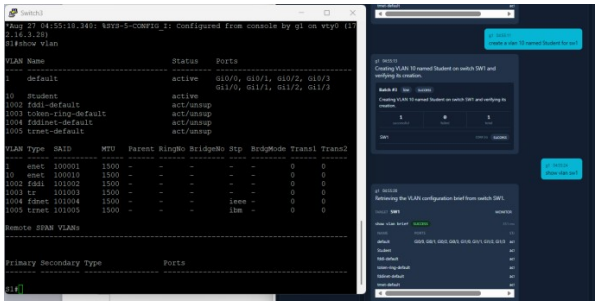


Figure 7. Retrieval of VLAN information from switch S1.

[Insert screenshot: chat request "xây 1 hệ thống mạng gồm 1 router, 1 switch, 6 PC, dải 172.20.5.0/24", the proposed diagram, and the topology canvas after import.]

Figure 8. Designing a network from a plain-language request.

Figure 8 shows a design request. The message "build a network with one router, one switch connected to the router and six PCs, using 172.20.5.0/24" is routed to the design advisor, which proposes VLANs, the router's interface addresses and the PC addresses. A follow-up such as "add another switch connected to the router" stays in the design conversation. After the user accepts the proposal, the diagram is imported into the canvas, where it can be edited, exported to PNETLab and bootstrapped.

[Insert screenshot: PNETLab export confirmation (nodes, Cloud0 ports) and the per-device bootstrap results; optionally the new lab in the PNETLab UI.]

Figure 9. Exporting a project to PNETLab and bootstrapping SSH.

Figure 9 shows the export of the project as a new PNETLab lab. The system creates the nodes and links, starts the nodes, and configures the management address and SSH on each device through its console; the results table lists each device, its Cloud0 port and whether SSH answered. [Report the number of nodes, total time and success rate here.]

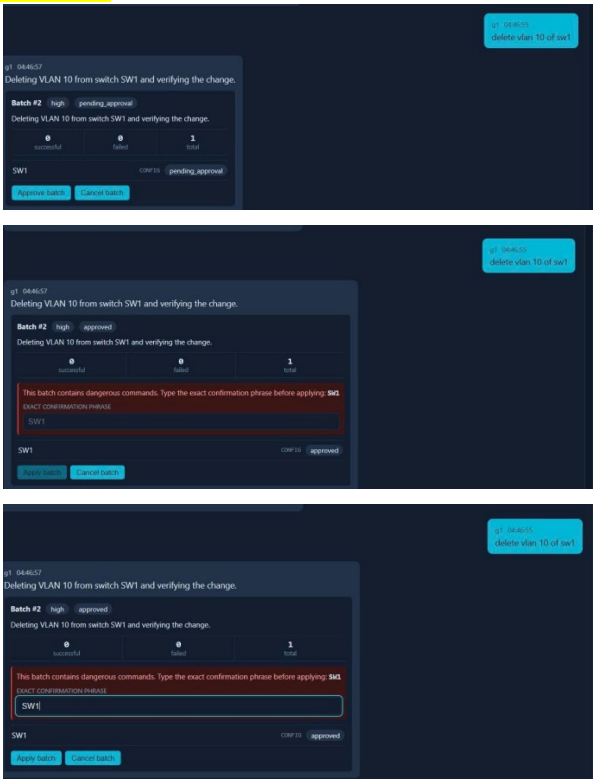


Figure 10. Deleting a VLAN: a high-risk change with typed confirmation.

Figure 10 shows a high-risk change. Deleting a VLAN is classified as High, so after approval the user must type the exact device name before the change is applied. The backend then backs up the

running configuration, applies the command and verifies the result. Typing the hostname forces the user to check the target: deleting VLAN 10 instead of VLAN 20 could disrupt the network.

[Insert screenshots: (a) a Student's preview waiting for approval, (b) the Teacher's approval, (c) a Student without a deployment grant being refused; and the checkpoint panel with the generated restore preview.]

Figure 11. Role-based approval and checkpoint-based undo.

Figure 11 shows the governance workflow. A student creates a configuration for an assignment device; the preview waits until the teacher approves it, and the student can then deploy it because they hold a deployment grant. A student without that grant is refused before any SSH session is opened. The same figure shows the undo of an applied change: the checkpoint produces a restore preview that is approved and applied like any other change.

C. Quantitative Evaluation

The evaluation measures four aspects. **Correctness of the implementation:** the backend test suite, which uses fake SSH and AI providers so that no device or model is contacted, passes 1,930 tests in 483 seconds; it covers authentication and token revocation, scoped grants and delegation, command policy, parsers, the AI schemas, change and batch workflows, verification, checkpoints, PNETLab import/export and bootstrap, and database integrity. **Intent routing:** the router is evaluated on [N] messages from Vietnamese and English conversations, labelled design, read, configure or chat, and compared with the keyword rules alone. **Structured actions:** a corpus of 50 cases in eight categories (switching and interfaces, static routes, ACL/DHCP/OSPF, monitoring, troubleshooting, chat, dangerous or unauthorised requests, and ambiguous requests) measures schema validity, intent and target accuracy and command match. **Safety:** for every case the evaluation counts SSH sessions opened during the AI request and commands that reach a device without approval. Table 4 summarises the results.

Table 4. Quantitative evaluation results.

Metric	Result	Source
Automated tests passed	1,930 (483 s)	pytest
Intent router accuracy (4 classes)	[X%]	eval_intent _router
Keyword rules accuracy (baseline)	[X%]	same set
AIAction schema validity	[X] / 50	evaluate_ai
Intent / target accuracy	[X% / X%]	evaluate_ai
Valid designs (1st try / after retry)	[X% / X%]	[20] requests
SSH opened during an AI request	0	evaluate_ai
Unapproved commands reaching a device	0	tests, corpus
Median latency: route / read / apply+verify	[X / X / X s]	traces

[Describe the results: where the router and the model err (for example, short follow-ups or mixed-language messages), how often the design advisor needs its second attempt, and the latency of SSH operations compared with model calls.]

D. Discussion

Compared with chatbot studies in networking education (Ahmed & Hasnine, 2023; Soto et al., 2024), NetAI grounds its answers in the state of the students' own devices. Compared with configuration systems such as NetConfEval, LLM-NetCFG and Intent-LLM (Wang et al., 2024; Lira et al., 2025; Provvedi et al., 2026), it keeps a human in the loop for every write, separates the roles of instructors and students, and makes each applied change reversible through a reviewed restore. The main lesson of the implementation is that the LLM is most useful for understanding intent and drafting proposals, while safety must come from deterministic backend checks: risk, permission and approval are never taken from the model's output. Routing with a model also handled conversational follow-ups that keyword rules misclassified, which is common in student chats that mix Vietnamese and English.

IV. CONCLUSION

This paper presented NetAI, an AI-assisted network management system for networking education. The system routes chat messages with an LLM, turns plain-language requests into validated designs,

exports them as PNETLab labs that it bootstraps over the console, and manages Cisco IOS, ASA and Palo Alto devices over SSH. Every configuration change is classified by risk, approved according to role-based grants, backed up, verified, audited and can be undone through a reviewed checkpoint, while the model never receives credentials or management addresses. The implementation passes 1,930 automated tests, and the evaluation [shows X].

The current system has limitations. Rollback is never automatic; generated OSPF uses a single area; CDP/LLDP discovery supports only Cisco IOS, and ASA devices must be bootstrapped by hand; SQLite and in-memory bootstrap jobs limit the deployment to a single server process; and PNETLab allows only one session per account. [The evaluation has not yet included a study with students.] Future work will add vendors through Netmiko, multi-area OSPF and BGP, an isolated simulator to test changes before deployment, automatic grading by comparing device state with the assignment, and a classroom study of the system's effect on learning.

REFERENCES

Ahmed, M. M. H., & Hasnine, M. N. (2023). Improving essential knowledge and self-efficacy in computers network course: The potential of chatbots. Procedia Computer Science, 225, 3929–3937. https://doi.org/10.1016/j.procs.2023.10.388

Anthropic. (2026). Claude API documentation. https://docs.anthropic.com

Budianto, A., Ariyuana, R., & Puspitaningrum, A. A. (2019). Intelligent tutoring system design for computer network learning. In Proceedings of the 2nd International Conference on Vocational Education and Training (ICOVET 2018) (pp. 75–80). https://doi.org/10.2991/icovet-18.2019.20

Byers, K. (2026). Netmiko documentation. https://github.com/ktbyers/netmiko

Cisco Systems, Inc. (2026). Cisco IOS configuration fundamentals configuration guide. https://www.cisco.com/c/en/us/support/ios-nxos-software/index.html

Colvin, S. (2026). Pydantic documentation. https://docs.pydantic.dev

Forcier, J. (2026). Paramiko documentation. https://www.paramiko.org

Google. (2026). Gemini API documentation. https://ai.google.dev/gemini-api/docs

Grinberg, M. (2026). Flask-Migrate documentation. https://flask-migrate.readthedocs.io

Gronholm, A. (2026). APScheduler documentation. https://apscheduler.readthedocs.io

Jiang, J., Jin, S., Li, X., Zhang, K., & Sun, B. (2025). A zero-touch dynamic configuration management framework for time-sensitive networking (TSN). Entropy, 27(6), Article 584. https://doi.org/10.3390/e27060584

Kim, K.-H., Park, C., Lee, H., Park, C., Kim, T., Choi, M.-J., & Kim, E. (2026). Towards fully autonomous network management: A survey on LLM-based multi-agent systems. ICT Express, 12(4), 1015–1034. https://doi.org/10.1016/j.icte.2026.07.003

Lira, O. G., Caicedo, O. M., & da Fonseca, N. L. S. (2025). Large language models for zero touch network configuration management. IEEE Communications Magazine, 63(7), 146–153. https://doi.org/10.1109/MCOM.001.2400368

Md Enzai, N. I. (2024). Utilizing ChatGPT for teaching computer networking. EDUCATUM Journal of Science, Mathematics and Technology, 12(1), 52–59. https://doi.org/10.37134/ejsmt.vol12.1.7.2025

NAPALM Automation. (2026). NAPALM: Network automation and programmability abstraction layer with multivendor support. GitHub. https://github.com/napalm-automation/napalm

Palo Alto Networks. (2026). PAN-OS CLI quick start. https://docs.paloaltonetworks.com

Pallets. (2026). Flask documentation. https://flask.palletsprojects.com

PNETLab. (2026). PNETLab documentation. https://pnetlab.com

Porzio, C. (2026). Alpine.js documentation. https://alpinejs.dev

Provvedi, C., Seidenari, L., Picano, B., & Fantacci, R. (2026). Intent-LLM: A framework for automated network configuration through code generation. IEEE Transactions on Cognitive Communications and Networking, 12, 7246–7258. https://doi.org/10.1109/TCCN.2026.3683230

Python Software Foundation. (2026). Python 3 documentation. https://docs.python.org/3/

Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Soto, D., Higashida, M., Shirai, S., Ueda, M., & Uranishi, Y. (2024). Enhancing learning dynamics: Integrating interactive learning environments and ChatGPT for computer networking lessons. Procedia Computer Science, 246, 3595–3604. https://doi.org/10.1016/j.procs.2024.09.198

SQLAlchemy. (2026). SQLAlchemy documentation. https://docs.sqlalchemy.org

SQLite Consortium. (2026). SQLite documentation. https://www.sqlite.org/docs.html

Verdú, E., Regueras, L. M., Gal, E., de Castro, J. P., Verdú, M. J., & Kohen-Vacs, D. (2017). Integration of an intelligent tutoring system in a course of computer network design. Educational Technology Research and Development, 65(3), 653–677. https://doi.org/10.1007/s11423-016-9503-0

Wang, C., Scazzariello, M., Farshin, A., Ferlin, S., Kostić, D., & Chiesa, M. (2024). NetConfEval: Can LLMs facilitate network configuration? Proceedings of the ACM on Networking, 2(CoNEXT2), Article 7. https://doi.org/10.1145/3656296